CREATING A UNIVERSAL INFORMATION LANGUAGE

OSCAR BENDER-STONE

ABSTRACT. Welkin is a formalized information language. We introduce its use cases and rigorously define its syntax and semantics. From there, we introduce the bootstrap, making Welkin completely self-contained. [TODO: determine how to phrase soundness and incompleteness. Should we include these?].

CONTENTS

1. Introduction . . . 2
2. Rationale . . . 5
   2.1. General Requirements . . . 5
   2.2. Units . . . 5
   2.3. Mechanizing Information . . . 6
   2.4. Base Rules . . . 6
   2.5. Base Operations . . . 7
   2.6. Bootstrapping . . . 8
3. Foundations . . . 9
   3.1. Base Rules . . . 9
   3.2. The Unit Recursor . . . 14
   3.3. Base Verifier . . . 16
   3.4. Queries and Information . . . 17
4. Syntax . . . 17
   4.1. Numbers . . . 18
   4.2. Encoding . . . 18
   4.3. Invertible Syntax Descriptions . . . 18
   4.4. Character Classes . . . 20
   4.5. The Welkin Grammar . . . 21
   4.6. Proof of Unambiguity . . . 21
   4.7. Import Resolution . . . 22
5. Metatheory . . . 22
   5.1. Establishing the Floor: $I\Delta_0$ . . . 22
   5.2. Artemov's Approach . . . 24
   5.3. Truth and Soundness . . . 25
   5.4. The Meta Unit . . . 26
   5.5. Proof of Complete Expressivity . . . 26
6. Conclusion . . . 27
   6.1. Comparisons to Existing Literature . . . 28
   6.2. Programming Languages and Formal Verification . . . 28
   6.3. Mathematical and Scientific Knowledge . . . 28
   6.4. Humanities . . . 29
A Bootstrap Document: `welkin.welkin` . . . 29
References . . . 31

DRAFT

This thesis used Google Gemini (https://gemini.google.com/) to aid in research and help fix formatting. All ideas presented come from the author.

Date: March 6, 2026.

1. Introduction

[TODO[MEDIUM]: determine if the phrase: "see Section ?" should be used, or "refer to" instead, to fully include all readers.]

Information Management is the study of systematically storing and organizing data. With the massive growth of data, particularly on the web, simply storing data to *use* it. Raw databases are increasingly structured into organized *information bases* to prove context surrounding the source and nature of data. The conceptual foundation for this process often traces back to Ackoff [Ack89]. His data-to-wisdom hierarchy posits that information is the process of transforming raw data into structured content. In turn, this content can be used to answer user queries. However, the term "information" has not been standardized in the literature and is used with distinct meanings. For example, Buckland [Buc91] recognizes three different ways information is interpreted: as a *process*, as *knowledge*, and as *a thing*. While each of these are important, Buckland emphasizes that to *use* information, one must work with information as a thing—a tangible record like a document or digital file. Another author, Floridi, builds upon this tangible idea by requiring data be truthful [Flo10]. Information must not only be in concrete records, but also accurately represent a topic of interest. As shown in this small sample of authors, information is a rich term, with an active pursuits to best manage it.

Despite these extensive ideas, applying Information Management in real world projects reveals ongoing problems. One of these issues is the *depth* of domains. For example, at major retailers like Amazon, inventory and recommendation systems use immense amounts of data to power personalized recommendations to users [SL17]. Another issue is the immense breadth of different subjects. As a well known example, Wikipedia struggles to constantly update its vast and interconnected web of knowledge [Mes+15]. Along with the depth and breadth of different disciplines, standardization is complex. Most systems have their own unique data formats, so making **interoperable** formats is hard, which is the ability to transfer one data format to another. For instance, transferring information between two medical platforms is notoriously difficult, even if they store the *same* information about a patient [Rei17]. These problems continue to be a challenge in effectively utilizing frameworks for information.

In consideration of these issues, several solutions partially resolve some of them, but not completely. The following provides a small list of major existing solutions. Reading these is optional and not required to understand this thesis. We will not revisit these until the conclusion, see Section 6.1.

- **Resource Description Framework (RDF) Triple Stores:** These systems serve as the primary implementations of an internet standard called the Web Ontology Language (OWL) [Hit+09]. This system stores information as triples $\text{subject} - \text{predicate} \rightarrow \text{object}$. From there, the language can enforce rules and ensure new relationships are derived *only* from from previous ones. OWL has been highly successful in providing a machine-readable format to websites across the internet [Hit21]. However, despite their widespread adoption, RDF stores struggle with extensive sources of data. Enforcing logical rules can be expensive [Zhu+23]. Additionally, because OWL treats missing sources of data as *unknown* rather than *false*, it often fails to resolve conflicts between two sources [Hit21].

- **Labeled Property Graphs (LPGs):** This is an architecture defined by its ability to store rich, internal metadata within its components. Building on a network, consisting of nodes and edges between them, LPGs enable both nodes (entities) and edges (relationships) to possess a set of key-value pairs known as *properties* [Ang18]. The internal metadata allows systems to efficiently traverse networks, as attributes can be filtered and weighed on a single query pass. Due to their operational efficiency, LPGs are the primary choice in large scale commercial products, particularly real-time fraud detection tools and recommendation systems [RWE15]. Although LPGs are widely used, they are siloed by the lack of standards, making interoperability between databases challenging. Furthermore, LPGs lack an extensive logical engine, resulting in external tooling or complex query scripts [RWE15].
- **Cyc:** Designed to encompass broad subjects, Cyc is the largest proprietary database of knowledge, or *knowledge base*, known to date [Len95]. It achieves reliable queries by providing a logical engine, based on millions of hard coded "common sense" rules. However, despite its impressive scope, it relies on human experts to manually encode every rule. This limits how well the database scales [Har07]. Another issue is the absence of an active open standard, a result of the publicly available version, OpenCyc, being discontinued in 2017 [KBp].
- **fcaR (Formal Concept Analysis in R):** This tool is a modern implementation for the field of Formal Concept Analysis (FCA). FCA studies relationships through *formal concepts*. The central idea is the *concept lattice*, a hierarchical data structure that maps objects to attributes in a fixed domain. Given only a database, fcaR can automatically identify the underlying core concepts and derived concepts [MDV26]. While being highly effective in specialized domains, concept lattices can become impractically large as the number of attributes increases [HS13]. This makes it difficult to interpret results when applied to large-scale databases or when bridging disparate domains [HS13].

This thesis introduces a language to resolve these issues for Information Management. I call this language **Welkin**, based on an old German word meaning cloud [Dic25]. This word represents the expansive nature of the language.

The core result of this thesis is proving that Welkin fulfills three goals: it is **universal**, **scalable**, and **standardized**. For details, see Table 1. The fundamental building block is based on **representations**, containing a relationships between a **sign** representing a **sign** in a **context**.[1] Contexts provide flexibility to define domain terminology, and are provided rules inspired by McCarthy's work in AI systems [McC93]. Moreover, truth is defined relative to contexts and enables the user to recognize *under which conditions* a statement is true. In this way, the information base keeps conflicting claims contained, providing a clear separation from other contexts.

Building on representations, information is defined as a property preserved under equivalences relative to a context. To make the information base scalable, the base operations are kept as simple as possible, and a procedure to adaptively organize information. The simplicity of the language aids in standardization as well. For the

[1]There are similarities with representations and Peirce's semiotics. This theory studies the relationship between a symbol, the object it represents, and the interpretation that provides that meaning [Atk23]. Our notion is different in that contexts are a generalized interpretant, see Section 2.2.

highest reassurance, we create a restricted, finite subset of the language. This subset facilitates proofs in this thesis and enables ease of implementation.

Goal	Name	Description
Goal 1	**Universality**	The language must enable any user created parameters, whose symbolic representation is accepted a computable function. Every computable function must be definable in the language.
Goal 2	**Scalability**	The database must appropriately scale to broad representations of information. Local queries must be efficient. Additionally, any future, optimizations that correctly implement the language should be expressible in the language.
Goal 3	**Standardization**	The language needs a rigorous and formal specification. Moreover, the bootstrap must be formalized, as well as an abstract machine model. The grammar and bootstrap must be fixed to ensure complete forwards and backwards compatibility. Certificates must be reliably checked and rely on a low level of trust, or a small Trusted Computing Base.

TABLE 1. Goals for the Welkin language.

This thesis is organized according to Table 2.

Section Number	Title	Description
Section 2	**Rationale**	Introduces Welkin at a high level, with guiding examples.
Section 3	**Foundations**	Introduces the core theory behind Welkin, which is the starting point to specify the user-facing language.
Section 4	**Syntax**	Provides the grammar and proof that it is unambiguous.
Section 5	**Metatheory**	Proves that Welkin can express any proof.
Section 6	**Conclusion**	Concludes with comparisons to existing theories and possible applications, particularly in programming languages and broader academic knowledge management.

TABLE 2. Organization for the thesis.

DRAFT

2. Rationale

In this section, we justify the design of Welkin.

2.1. General Requirements.

The main purpose of an information base is to store information and answer user queries based on available information. To provide an upper bound on our requirements, we require two important properties:

1) The information base must be mechanized, which means only "effective operations" are allowed. We make "effective operations" precise with the technical notion of *partial computable functions*. Partial computable functions are computer programs that can process a finite input, and that either stop on an input or continue forever. The bedrock for these functions is the notion of a *Turing machine*, which models every current form of computation. For more information, see [Tur36]. To obtain universality, we need to express *every* one of these machines.
2) The information base must provide a mechanism to *ground* symbols defined by users. This stems from the Symbol Grounding Problem, posed by Harnard [Har90]. To avoid storing physical objects or such, users should have a way to provide *meaning* to symbols, or a way to *represent* their topic of interest.

Combined together, we can now state universality as follows: *an information base is universal if it can represent anything representable by a partial computable function.* We will clarify how we define "representable" in the next section.

2.2. Units.

[TODO[MEDIUM]: double check all examples!]

The first core concept in Welkin is the notion of a **handle**. Handles are left as free parameters in the theory[2] In an information base, handles are uniquely identified through their **key**, directly inspired by database theory. Keys are with triples of IDs $(\mathrm{MID}, \mathrm{RID}, \mathrm{SID})$, whose contents are the **origin ID**, **module ID**, **revision ID**, and **handle ID**, respectively. Each ID serves the following purpose:

- The **module ID** MID defines the ID for a given string written in Welkin. This identifies the **parent** of the contents of a string.
- The **revision ID** RID ensures that storing specifications is **immmutable**, or provide a static snapshot of a unit. This is *not* a restriction on representing dynamic entities, but rather, this indicates to the information base which version of a handle to use.
- The **symbol ID** SYM refers to the symbol used to index the handle.

We illustrate the use of these IDs in an example.

Example 2.2.0. In a scientific experiment, a handle could be an observation or experimental data. The unit is then written as a symbol, say u, and is *implicitly* bound to this meaning. To distinguish from other symbols, say v, the computational content is analyzed.[TODO[MEDIUM]: expand out this example!]

Indexing handles is only one important aspect of handles. Another is that handles are precisely by their relationships. These are written as **representations** $a - c \to b$,

[2]For mathematicians, this is analogous to how key notions are left undefined in Hilbert's formalism of geometry, or how a "set" is an undefined notion in set theory.

DRAFT

which means that a represents b in context c. Here, we call a the **sign**, b the **referent**, and c the **context**. One can interpret this characterization as follows: handles are defined by how they are and are *not* restricted. This idea is directly inspired from Fine's idea of arbitrary objects [Fin85], and allows refinements to a study of interest.

Example 2.2.1. A more looser example is a user written journal for therapy sessions, containing information about daily habits and emotions. While neither of these are stored in the information base, their handles are, via units habit and $\mathrm{emotions}$ in a context $\mathrm{journal}$. Moreover, multiple revisions of the journal can be made with dates or other unique IDs.

We represent handles in Welkin using an extended notion called a **unit**. A unit is either a handle, a finite collection of units, or a representation. Collections of units can have a uniquely generated name by the information base called a **block**[3].

Example 2.2.2.

Units define their own **context**, which means the unit provides its own names and representations. Writing $a - c \to b$, therefore, only applies to the *fixed* context c.

[TODO[SMALL]: use better names for entities/businesses/etc]

Example 2.2.3. A business could represent their operations using a unit $\mathrm{business}$ that contains units for their workers and ledgers. This allows another unit, say $\mathrm{business2}$, to contain its *own* label $\mathrm{workers}$ that is separate from the one in $\mathrm{business}$. In addition to separate labels, these contexts can have *distinct* rules, such as those for how business operations are performed.

2.3. Mechanizing Information.

From the notion of a unit, we practically define information on a unit v as a *unit that tracks reductions in* v. Formally, we define information as a proof used in derivations, which include refinements of v to other units. Key to this definition is making it *context-based*, thereby enabling separate truth-management systems (those systems defined by some Turing machine). This notion corresponds to Burgin's idea of information as an *operator* that transforms a system, and is closely to Bateson's famous quote that "information is a difference that makes a difference" [Bat00]. Our practical distinction between information and knowledge is that we *use* information, but users can assert their own notions of these terms by creating restricted contexts. There are technical constructions to show that our definition is *as general* as possible (Definition 3.4.18).

2.4. Base Rules.

There are two primary inference rules, which we informally describe now:

- **Internal Transitivity:** if u represents v in context c and v represents x in context c, then u represents x in context c.
- **Lifting:** if u represents v in context c and p represents q in context v, then within context c, p represents q in *nested* context u.

Transitivity is a common axiom in many systems. It is particularly efficient when used in contexts, as shown by several projects [Len+90], [GWW07]. Lifting is a notion

[3]In programming languages, this is analogous to an anonymous function.

DRAFT

closely tied to McCarthy's notion of lifting, but formulated with representations. The idea is, if a represents b in context c, then a provides an *abstraction* for b relative to c. Wherever b is involved in a relationship, a can be used *instead*, acting as a proxy relative to c. Contexts provide an import mechanism as well, thereby providing Welkin with a module based system. We provide a simplified version of one McCarthy's examples below. For more existing examples in the literature, refer to [BF95].

[TODO: clean up further!]

Example 2.4.4. (McCarthy's Above Theory). Consider a set of physical blocks. Denote the unit above to represent the relation: *block* A *is above block* B. We want to say this could be on to mean: *block* A *is on block* B, or $\mathrm{floating_above}$ to mean: "block A is floating above block A. One way to state is as two axioms: . Alternatively, we could state as one axiom: $\mathrm{A} - \mathrm{on} \to \mathrm{B}$ implies $\mathrm{A} - \mathrm{above} \to \mathrm{B}$ and $\mathrm{A} - \mathrm{floating_above} \to \mathrm{B}$ implies $\mathrm{A} - \mathrm{floating_above} \to \mathrm{B}$. Lifting provides a more economical approach *without* needing to provide explicit terms, or provides a "point-free" rendering. We could that, within above, above represents on and above represents $\mathrm{floating_above}$. In other words, above *precisely abstracts* from the precise relationship between the two blocks.

Another example concerns contexts in the presence of unique objects.

Example 2.4.5. Consider a biological survey of home pets, denoted by context Survey. Suppose there are two units in Mammal, say a dog Fido and a cat Lucy. We could state that Fido *represents* Mammal, to say that Fido acts as a stand-in for a mammal in the survey. The same could be said with Lucy representing Mammal. Note that, depending on the context, the referent need *not* be more refined than the sign; this is an intenional design choice for flexibility. With a another context $\mathrm{Taxonomy}$, we may naturally state that $\mathrm{Mammal} - \mathrm{Taxonomy} \to \mathrm{Animal}$, and we could say $\mathrm{Survey} - \mathrm{instanceof} \to \mathrm{Taxonomy}$. Thus, the lifting rule implies in context $\mathrm{instanceof}$, $\mathrm{Mammal} - \mathrm{Survey} \to \mathrm{Animal}$. Using transitivity, we obtain $\mathrm{Fido} - \mathrm{Survey} \to \mathrm{Animal}$ and $\mathrm{Lucy} - \mathrm{Survey} \to \mathrm{Animal}$ in context $\mathrm{instanceof}$. We can interpret this example as saying that the *relationships* of a general taxonomy are witnessed by a specific taxonomy, namely Survey, and that they propagate through contexts.[4]

The remaining rules are primarily for efficiency, enabling users to have positionally invariant information, see Definition 3.1.11.

2.5. **Base Operations.**

Now, units and information themselves could be expressed in infinitely many languages, with slightly different syntax or semantics. Welkin is carefully designed to be a *minimal* expression of these concepts, with minimal friction to express any other universal information base. We write these in ASCII in `type-writer` font to show that they are shown digitally. These include:

- In ASCII, `a - b -> c` for representations. These can be interpreted *as* rewrite rules, depending on the context.

[4]In programming languages, particularly C++ [Str13] and Java [AGH05], this property is known as "upcasting", with a less refined base class being replaced with a more refined subclass. The reverse is "downcasting". Both are supported by Welkin for full expressivity.

DRAFT

- Traditional braces `{ }` to denote closed definitions of contexts, inspired by the C programming language.
- Paths via dots `.` that is inspired by the Python programming language. Relative paths are denoted with multiple dots `...`, and absolute imports are prefixed with `#`. Subsets of units can be written via `u.{v, x}`, or even `u.{v --> x}` to refer to a subset of representations.
- Imports are done through `@u`, which takes all subunits of `u` and puts them into the current scope. In other words, the implementation *implicitly* adds denotations `v <--> a.v` for each subunit `v` of `u`.
- Selecting specific nested units can be done via `u.{v, x}` or, equivalently, `@u.{v, x}`.
- Imports can be *negated* via the notation `{@g, ~u}`. This is an uncommon feature of most programming languages, appearing primarily in Haskell and CSS. While potentially opaque, Welkin provides robust definitions to ensure that negated forms can be easily translated into more explicit ones *and* back again. [TODO[SMALL]: provide link, probably to bootstrap or so?]
- Comments *are* strings can be treated as any unit. No comments need to be removed in the files and can *enhance* the study of new subjects.[5] These units can *then* be analyzed by others to promote translations to other human languages, or be studied through the *overarching relationships* in the units.

In general, the minimal restricted keywords is crucial for providing support for other languages. An implementation of Welkin will contain a small section of ASCII encoding for easier standardization, but the rest of the program can be done *entirely* in the user's native language. This is a novel feature in most programming languages, which are either predominantly English or are fine tuned for specific human languages.

2.6. **Bootstrapping.**

There are two important considerations in Welkin: how the base operations are implemented, and how proofs are expressed. In previous revisions, the author separated these aspects, particularly when describing the syntax and semantics of the language. However, this left many proofs in English, making it difficult to transfer.

[TODO: provide citations] The author's emphasis of proofs stems from formal methods, the study of providing mathematical proof to verify computer programs. A central notion in this field is the **Trusted Computing Base (TCB)** of a program, which is code used to implement it, as well as the axioms used to prove its correctness. Many authors advocate for a small, fixed TCB. However, this is inefficient in practice… Setting for a single system, like ZFC, does not resolve whether Welkin is universal *for proofs*.

To resolve both issues, we "bootstrap" Welkin from the ground up.

- First, we show that Welkin can express any computer program, as well as a verifier for its own claims.
- Next, we define the syntax using the notion of **invertible syntax descriptions**, introduced in [RO10]. This will allow us to define the syntax *entirely* in Welkin.

[5] Contexts can mimic regular comments, but Welkin aims to be inspectable by users. Encapsulation or private information can be enforced through *rejecting* specific contexts or only accepting certain ones.

DRAFT

We will also prove the syntax is **unambiguous**, or that given any string, if it is accepted, there must be one way to process it in the program.
- Then, in Section 5, we prove that our definition of information is complete with respect to being accepted by a Turing machine.

3. Foundations

This section discuss the foundations of Welkin, as follows:

1) We define units and their rules.
2) We prove there is a unit that indexes every other unit (Section 3.2).
3) We define queries.

3.1. Base Rules.

As high-level notation, we write $a \equiv b$ to mean that a is definitionally equivalent to b. High level notation will *not* be stated in the syntax unless where noted. Moreover, we will not worry about *enforcing* a term is finite; this is left for implementations.

Definition 3.1.0. A **bit** is the symbols 0 or 1. A **binary word** is either the symbol ε (the **empty word**), and if w is a finite binary word, so are $w.0$ and $w.1$, where . is the symbol for **concatenation**. Nothing else is a word.

Definition 3.1.1. Equality $=$ and inequality $\neq$ on words w_1, w_2 is defined recursively:
- **Base case:** $\varepsilon = \varepsilon$.
- **Recursive step:** suppose $w_1 = b_1.w_{1'}$ and $w_2 = b_2.w_{2'}$, where b_1, b_2 as bits and $w_{1'}, w_{2'}$ are words. Then $w_1 = w_2$ if and only if $b_1 = b_2$ and $w_1' = w'2$. Moreover, $w_1 \neq w_2$ if and only if $b_1 \neq b_2$ or $w_1' \neq w_2'$.

Remark 3.1.2. Definition 3.1.8 is given constructively to ensure that if two finite words are unequal, then an explicit bit can act as a certificate for this inequality.

Words alone do not carry meaning. The extended meaning is provided to *handles*.

Definition 3.1.3. A *handle* is given by a **key**, a triple $(\mathrm{MID}, \mathrm{RID}, \mathrm{SYM})$, where MID is a binary word called a **user ID**, RID is a binary word called the **revision ID**, and SYM is a binary word called the **symbol ID**. The interpretation of keys are left as free parameters, and therefore are outside the scope of this language.

Remark 3.1.4. Because handles act as free parameters, we work with them through truth. Based on this, the rest of the thesis will abstract away general units *as* sets of axioms. Free parameters are used to define Welkin itself, however; see Section 4.

[TODO: decide if words are the ONLY things that automatically interpret fields, for simplicity. LIkely to be easier to do this **internally**]

Remark 3.1.5. For simplicity, we will *interpret* words as handles, internally generated by an implementation. The original value will be stored separately; see Section 4 for details.

[TODO[SMALL]: without getting into typing, enforce that equality has to be done on two words or two handles. Maybe lift to the latter to make sense?]

DRAFT

Definition 3.1.6. Consider two handles $h_1 \equiv (\mathrm{MID}_1, \mathrm{RID}_1, \mathrm{SYM}_1), h_2 \equiv (\mathrm{MID}_2, \mathrm{RID}_2, \mathrm{HID}_2)$.

- h_1 and h_2 are equal, written $h_1 = h_2$, if and only if $\mathrm{MID}_1 = \mathrm{MID}_2$, $\mathrm{RID}_1 = \mathrm{RID}_2$, and $\mathrm{SYM}_1 = \mathrm{SYM}_2$.
- h_1 is not equal to h_2, written $h_1 \neq h_2$, if and only if at least one of the following hold: $\mathrm{MID}_1 \neq \mathrm{MID}_2$, $\mathrm{RID}_1 \neq \mathrm{RID}_2$, or $\mathrm{SYM}_1 \neq \mathrm{SYM}_2$.

We reserve an important symbol as notation, used outside of the language: G for the **global context**. This can be interpreted as the combination of *all* possible units, with any string ID. For simplicity, this context does *not* have outside referents. To do this, we use special *reserved* units that do. We will discuss details later.

Now we can define units.

Definition 3.1.7. A **unit** is defined recursively as follows and nothing else:

- **Base case:**
  - A word, see Definition 3.1.7.
  - A handle, see Definition 3.1.9.
  - The symbol $\{\}$, the **empty block**.
- **Recursive step:** let u, v, g be units and h a handle. Then any finite combination of the following are also units:
  - $@u$, the **expansion** of u.
  - $\{u, v\}$, the **pair** of u, v.
  - An **import block**, defined as $\{@g\}$ for a unit g, or as $\{b, @u\}$, $\{b, \sim u\}$, or $\{\sim u, b\}$ for an import block b. In $\{b, \sim u\}$, $\sim u$ is the **exclusion** of u.
  - $g.v$, the **access of v on g**.
  - A **representation** $a - c \to b$, where a is the **sign**, c is the **context**, and b is the **referent**. This is read as: a **represents** b **in context** c.

We will intentionally *avoid* defining equality on units and postpone this until Section 3.2.

DRAFT

Definition 3.1.8. The following rules apply, stated over meta-variables a, b, c, d, g for units, w_1, w_2 for words, and h_1, h_2 for handles:

Rule	Name	Content
R1	**Internal Transitivity**	$a - c \to b$ and $b - c \to d$ imply $a - c \to d$.
R2	**Contextual Lifting**	$a - c \to b$ and $d - b \to g$ imply $\{d - a \to g\} - c \to \{d - a \to g\}$
R3	**Empty**	$\{@g, \{\}\} \longleftrightarrow g$
R4	**Null**	$\{a - \{\} \to b\} \longleftrightarrow \{\}$
R5	**Word Substitution**	If word w_1 is equal to word w_2, then $w_1 \longleftrightarrow w_2$.
R6	**Handle Substitution**	If handle h_1 is equal to handle h_2, then $h_1 \longleftrightarrow h_2$.
R7	**Identity**	$a \longleftrightarrow \{a - a \to a\}$
R8	**Singleton**	$a \longleftrightarrow \{a\}$.
R9	**Membership**	$\{@g, a\} \longleftrightarrow g$ if and only if $a - g \to a$.
R10	**Field Access**	$g.a \longleftrightarrow a$ if and only if $a - g \to a$.
R11	**Expansion**	if $a - g \to b$, then $\{@g, c\} \longleftrightarrow \{\{\{@g, a\}, b\}, c\}$
R12	**Exclusion**	if $g \longleftrightarrow \{@g, a\}$, then $\{\{@g, \sim a\}, b\} \longleftrightarrow \{@d, b\}$
R13	**Associativity**	$\{a, \{b, c\}\} \longleftrightarrow \{\{a, b\}, c\}$
R14	**Commutativity**	$\{a, b\} \longleftrightarrow \{b, a\}$ and
R15	**Import Commutativity**	$\{@g, \sim u\} \longleftrightarrow \{\sim u, @g\}$ and

We review the utility of each rule. Note that rules *between* contexts is entirely flexible and user defined. Moreover, only **R1** is needed for Turing completeness. However, the other rules are in place to help with organizing units as modules, as well as make it easier to use the language.

- **R1** and **R2** were discussed in Section 2.2.[TODO: maybe review the discussion from earlier? Might be useful to reinforce main ideas to reader.]
- **R3** and **R4** define the behavior of the empty unit $\{\}$, similar to the empty set. **R4** specifically states that $\{\}$ contains *no* representations, so any term $a - \{\} \to b$ may be written.
- **R5** and **R6** enables equality in words and handles to pass through into representations. Besides this, note that equivalences on units are *entirely* user defined.
- **R7** represents identity. This is *not* the same as $a - g \to a$, see the discussion below.
- **R8** reduces extraneous blocks. Note that this is *not* the same thing as the Quine atom, which states $\{a\} = a$ in a set theoretic context [Qui37]. We interpret $\{a\}$ as a *wrapper* around a. While not useful for handles, it is for specifying blocks of representations, such as $\{a - b \to c, b - c \to d\}$.

DRAFT

- **R9** provides a way to represent $a \in g$ through the representation $a - g \to a$. Because of this, $a - g \to a$ may be a non-trivial path, *not* identity.
- **R10** provides a mechanism to access specific units in a scope. The notation is entirely inspired by object oriented programming.
- **R11** and **R12** define how imports in the language work. An **import** is the process of joining the contents of one unit into another. **R11** says how an import can add new units in a block. **R12** provides a mechanism to *exclude* specified contents in g, which can themselves be representations. For example, $\{@\{a - b \to c, d\}, \sim\{a - b \to c\}\}$ reduces to $\{d\}$. This also enables a way to disable *any* rule in Welkin, based on user preference. This is intentionally restricted *per context*, so the rules are enacted by default. Also note that exclusions **must** be used in the presence of an expansion. Otherwise, terms like $\{a, \{b, \sim a\}\}$ would be allowed, so certain units could never be expanded!
- **R13**, **R14**, and **R15** ensure that information can be repeated and can be put into any order. Note that this does **not** mean $\{a, \{@g, \sim b\}\} \longleftrightarrow \{@g, \{a, \sim b\}\}$ for general units a, b. If this were the case, then one could write $\{a, \{@\{a\}, \sim a\}\}$ and produce $\{\}$, so extending certain units would be impossible!

We provide some reoccurring properties in the lemma below.

Lemma 3.1.9.

- *For every unit u, there exists a unit u' with at least one more entity than u. [TODO: make this more clear! This says units can always be extended to new ones!]*
- *Import blocks are locally associative. More precisely, given import blocks m_1, m_2 and units u, v, $\{m_1, \{m_2, \sim u\}\} \longleftrightarrow \{m_2, \{m_1, \sim u\}\}$.*

As more notation, we write:

- $a - c \to b_1 \mid b_2 \mid \ldots \mid b_n$ to mean $\{a - c \to b_1, a - c \to b_2, \ldots, a - c \to b_n\}$.
- $a_1 \mid \ldots \mid a_n - c \to d$ to mean $\{a_1 - c \to d, \ldots, a_n - c \to d\}$.

This simplifies the presentation of the rules. We postpone formally defining the operator $\mid$ to the syntax in Section 4. Moreover, we allow $c \longrightarrow \mid b_1 \mid b_2 \mid \ldots \mid b_n$, which is synonymou to $a - c \to b_1 \mid b_2 \mid \ldots \mid b_n$.

Before we proceed to prove Turing completeness, we introduce the SK-combinator calculus. This is an equational theory that is well known to be Turing complete [Cur30]. We provide a full definition as follows.

DRAFT

Definition 3.1.10. The SK-combinator calculus consists of the following:

- A **term** is defined recursively as either K or S, and if M, N are terms, so is (M) and their **application** MN.
- Evaluation: two terms M, N are **equal**, written $M = N$, if they can be deduced from the following axioms:
  - **Base Rules:** for all terms M, N, P:
    - $KMN = M$
    - $SMNP = MP(NP)$.
  - **Congruence:** if $M_1 = M_2$ and $M_2 = N = 2$, then $M_1 M_2 = N_1 N_2$.
  - **Equivalence:** $=$ forms an equivalence relation, which means for all M, N, P:
    - $M = M$.
    - $M = N$ if and only if $N = M$.
    - $M = N$ and $N = P$ imply $M = P$.

Theorem 3.1.11. *Any partial computable function can be expressed as some unit.*

Proof. We prove that we can embed any term in the SK-combinator calculus, defined in Definition 3.1.13. This proof includes an important technique to represent *recursion*,

We express recursion via a unit L. For keys, we set:

- $L \equiv (1, 0, 0)$.
- $K \equiv (1, 0, 1)$, $S \equiv (1, 0, 2)$.
- $M \equiv (1, 0, 3)$, $N \equiv (1, 0, 4)$ $P \equiv (1, 0, 5)$.

Note that these IDs will be reused after this lemma, as this just shows its *possible* to store into an information base.

For the rules, see Figure 1. Note that each rule of the form $A \longrightarrow B$ written in L means $A - L \to B$.[6]

$$
\begin{gathered}
L \equiv \{K \to L, \quad S \to L, \\
\{M \to L\} \to L, \\
\{N \to L\} \to L, \\
\{P \to L\} \to L, \\
C \leftrightarrow \{N - M \to L\}, \\
C \to \{M \to L\}, \\
C \to \{N \to L\}, \\
\{N - \{M - K \to L\} \to L\} \to \{M \to L\}, \\
\{P - \{N - \{M - S \to L\} \to L\} \to L\} \\
\to \{\{P - N \to L\} - \{P - M \to L\} \to L\}\}
\end{gathered}
\tag{1}
$$

FIGURE 1. Definition of L [TODO: fix formatting!].

This definition means:

[6]As a note for logicians: these rules are extremely similar to a Hilbert-style proof system, with K and S corresponding to the rules $(\varphi \Rightarrow (\psi \Rightarrow \varphi))$ and $(\varphi \Rightarrow (\psi \Rightarrow \zeta)) \Rightarrow ((\varphi \Rightarrow \zeta) \Rightarrow (\psi \Rightarrow \zeta))$, respectively. This was one of Curry's insights in connecting logic to computation [Cur30].

DRAFT

- L includes K and S as base cases. We interpret $K \longrightarrow L$ to mean K is a term of L.
- We include variables M, N, P over terms. The statement $\{M \longrightarrow L\} \longrightarrow L$ wraps around any term.
- Composition MN is represented as $N - M \to L$. Moreover, MN can be broken down into its constituent parts M, N.
- The remaining representations are for the rule of K and S, respectively.

[TODO: maybe present SK-calculus based on reductions, not equality? We can get equality easily through transitivity, but do discuss.]

Now, L already includes the base rules for K and S. It remains to be shown that L is closed under composition: $M \longrightarrow L$ and $N \longrightarrow L$ imply $C \longrightarrow L$, where $C \longleftrightarrow \{N - M \to L\}$. By using **R1** on $C \longrightarrow \{M \longrightarrow L\}$ and $\{M \longrightarrow L\} \longrightarrow L$, we obtain $C \longrightarrow L$, completing the proof. □

3.2. **The Unit Recursor.**

The proof of Theorem 3.1.2 demonstrates how contexts enable powerful recursive definitions. However, the underlying construction is tedious and results in extremely verbose terms. Keys are assigned manually, which can easily be error prone. We will refine the proof with a **recursor** over units. This is a unit that indexes every unit, as well as every handle. We gradually build unit based on notions for word and handle. Note that we will manually index new handles within the final bootstrap, see ?.

First, we need to establish some default units in the global context:

- The `welkin` unit. This is defined in Appendix A.

The rest of this thesis will take place in the `welkin` module except where noted.

Now, we present Figure 2.

$$\begin{gathered}\mathrm{bit} \longrightarrow 0 \mid 1,\\ \mathrm{word} \longleftrightarrow \{\} \mid \{\mathrm{top} \longrightarrow \mathrm{bit}, \mathrm{next} \longrightarrow \mathrm{word}\}\end{gathered}$$

FIGURE 2. Generator for words in Welkin.

This is similar to the Lisp definition of a list. In detail:

- bit represents 0 or 1.
- word is recursively defined, as either $\{\}$ (for empty) or the pair $\{\mathrm{top}, \mathrm{next}\}$, where top is a bit and next is a word.

From there, we can define handle IDs through triples, see Figure 3.

$$\mathrm{handle} \longleftrightarrow \{\mathrm{MID} \mid \mathrm{RID} \mid \mathrm{SYM} \longrightarrow \mathrm{word}\}$$

FIGURE 3. Generator for handle keys in Welkin.

Here, a handle is simply a pair $\{\mathrm{MID}, \mathrm{RID}, \mathrm{SYM}\}$, where $\mathrm{MID}, \mathrm{RID}, \mathrm{SYM}$ are words.

We need to include equality as well, refer to Definition 3.1.10:

DRAFT

$$
\begin{aligned}
\text{equals} \longleftrightarrow \{ & \\
0 \longleftrightarrow 0, & \\
1 \longleftrightarrow 1, & \\
\sim\{0 \longleftrightarrow 1\}, & \\
\text{w1} \mid \text{w2} \longrightarrow \text{word}, & \\
\{\text{w1} \longleftrightarrow \text{w2}\} \longleftrightarrow \{\text{w1} \longleftrightarrow \{\} \longleftrightarrow \text{w2}\} & \\
\mid \{\text{w1.top} \longleftrightarrow \text{w2.top}, \text{w1.next} \longleftrightarrow \text{w2.next}\}, & \\
\text{h1} \mid \text{h2} \longrightarrow \text{handle}, & \\
\{\text{h1} \longleftrightarrow \text{h2}\} \longleftrightarrow \{ & \\
\text{h1.MID} \longleftrightarrow \text{h2.MID}, & \\
\text{h1.RID} \longleftrightarrow \text{h2.RID} & \\
\text{h1.SYM} \longleftrightarrow \text{h2.SYM} & \\
\}\} &
\end{aligned}
$$

FIGURE 4. Definitions of equality in Welkin.

To show these constructions are correct, we must prove the following.

Lemma 3.2.12. *Let* h1 *and* h2 *be handles. Then* $\text{h1} \leftarrow \text{equals} \rightarrow \text{h2}$ *if and only if* $h_1 = h_2$.

Proof. Clearly it is sufficient to show that equality on words is correct. To do so, we apply a simple proof by induction:

- **Base case:** immediate.
- **Inductive step:** correctly handles the the cases for top and next.

□

Now, in Definition 3.1.12, we needed enough *separate* meta-variables. To do this in Welkin, we use representations of the form $\text{u} \longrightarrow \text{unit}$. This appeared frequently when defining terms in Theorem 3.1.2.[7]

Definition 3.2.13. The **unit recursor** unit includes all rules in Definition 3.1.12, as well as the following in context unit:

- $\text{word} \longrightarrow \text{unit}$, see Figure 2.
- $\text{handle} \longrightarrow \text{unit}$, see Figure 3.
- $\{\} \mid u \mid v \mid c \longrightarrow \text{unit}$.
- $@u \longrightarrow \text{unit}$.
- $\{u, v\} \mid \{@u, \sim v\} \mid \{\sim v, @u\} \longrightarrow \text{unit}$.
- $\{u - c \rightarrow v\} \longrightarrow \text{unit}$.

The following statements are two parts of the same **Recursion theorem** for Welkin. The first is straightforward; their proofs closely aligns with the definitions written in the meta-language (English).

Lemma 3.2.14. (Recursor Correctness). *For every unit* u, $u - \text{unit} \rightarrow \text{unit}$.

[7][TODO: discuss how we are allowing the variables to be the same or different! Related to bundling in MetaMath + MetaMath Zero [and cite these!].]

DRAFT

Proof. Fix the context to be unit. We proceed induction on units:

- **Base case:** this is immediate, as $\{\}$ and all handles are included.
- **Inductive step:** immediate; unit includes representations $a - b \to c$, as well as cases for $@u$, $\{u, v\}$, $\{@u, \sim v\}$, and $\{\sim v, @u\}$.

□

Now, for the rest of the thesis (except bootstrap), we will drop mentions of specific handles. These will all be facilitated by unit. Moreover, if $u \longrightarrow \mathrm{unit}$, then $u - \mathrm{unit} \to \mathrm{unit}$, based on **R7** and **R1**. We will use the former for brevity.

In addition to `unit`, we need a mechanism to assign IDs for two things: words and blocks. For the latter, we need a way to normalize at least to index those groups that can be made associative.

3.3. **Base Verifier.**

Now we will includle a notion for containment. Because Welkin is Turing expressible, unit may not terminate in all cases, such as an infinite recursive loop. We want to have a mechanism to *check* certain claims. This is the role of the verifier, built upon part.

[TODO: start converting list defs into single equations.]

Definition 3.3.15. The unit part is defined over units $u, v \longrightarrow \mathrm{unit}$:

- $u \longrightarrow \{@u, v\}$.
- $\sim\{\{@u, v\} \longrightarrow u\}$.
- $\{@u, \sim v\} \longrightarrow u$.
- $\sim\{u \longrightarrow \{@u, \sim v\}\}$.

Lemma 3.3.16. $u - \mathrm{part} \to u'$ *if and only if* $u - u' \to u$.

Proof. We proceed by structural induction on units, or $u \mid u' - \mathrm{unit} \to \mathrm{unit}$:

- **Base case:** we need to prove $\{\{\} - \mathrm{part} \to u'$ if and only if $\{\} - u' \to \{\}$.
  - **Base case:** For $u' \equiv \{\}$, clearly both are false.
  - **Inductive step:** There are three cases:
    - **Representations:** immediate.
    - **Expansions:** .
    - **Exclusions:** .
- **Inductive step:** similar to the base case. [TODO: complete this proof! But it is straightforward.]

□

We can now define the verifier entirely in Welkin. Add new handles accept and reject.

[TODO: deal with recursion! Need to ensure it halts! And make sure to establish a clear induction scheme! And maybe add a pair to store the context and target? Fill in _!]

DRAFT

Definition 3.3.17. The unit verify is defined over a context c, a unit $\{a - d \to b\}$ called the **target**, and a unit p called a **derivation**. We define this by recursion on p.

- **Base case:** if p, $\{\mathrm{context} \longrightarrow c, \mathrm{target} \longrightarrow \{a - d \to b\}, \mathrm{derivation} \longrightarrow p\} - \mathrm{verify} \to \mathrm{accepts}$ holds if and only if $\{a - d \to b\} \leftarrow \mathrm{part} \to p$. Similarly, reject if and only if $\sim\{\{a - d \to b\} - \mathrm{part} \to p\}$.
- **Recursive step:**. suppose $p \equiv \{p', e - f \to g\}$. Then $\{p_{,\}} - \mathrm{verify} \to \mathrm{accept}$ if and only if p' is accepted in context . [TODO: get the end of p' and wrap it up in the context and target. See if that's accepted. Then, add on an extra step and see if it's valid.]

Lemma 3.3.18. *For all contexts c, targets $\{a - b \to d\}$, and derivations p,* verifier *is* ***sound****; it correctly identifies if p is a derivation of $\{a - b \to d\}$ in c.*

Proof. TBD[FINISH full inductive proof!] □

Note that this verifier, as simple as it is, will *not* limit what proofs can be expressed. We prove this in Theorem 5.5.16.

3.4. Queries and Information.

By Theorem 3.1.2, every partial computable can be expressed as a unit. Additionally, in the construction used, reductions of terms are *also* represented. This provides a ceiling on what queries we can *express* computably. For more details, refer to [HMU07]. This asks whether a representation is contained in a context.

Definition 3.4.19. Let c be a context. Then a **query over c** is a question over fixed units p, q, d: "Does $\{p - q \to d\} - \mathrm{unit} \to c$ hold?"

We can then apply unit in an attempt to evaluate this. In some cases, this is easy and only uses *direct definitions*. Consider, for example, examining p in context $\{p, q\}$. We can resolve this with a quick traversal, showing that p is in fact in $\{p, q\}$. However, Theorem 3.1.2 implies that finding general derivations are uncomputable.

We want information to *resolve* a query. Formally, we want information to be *proof*, not a "yes/no" answer by itself. When the context is finite, like $\{p, q\}$, there is a direct proof. But what about the general case? Some queries may reduce *forever*. While we cannot determine *which* ones reduce to $\{\}$ or not, we *can* find the ceiling of proofs expressible by computable functions. This process is more involved and is the subject of the next section.

[TODO[MEDIUM]: make this definition self-contained! The goal is to show that this *is* complete. That should be independent of the next section!]

Definition 3.4.20. Information about a query q in context c is a any partial meta-proof in meta of a derivation of $\{@c, q\}$.

4. Syntax

[TODO[SMALL]: determine where to use type writer font. Maybe ONLY in the bootstrap?] For consistency with Welkin, we write syntax using `type-writer font`.[8]

[8]This font is Intel One Mono (https://github.com/intel/intel-one-mono), licensed under OFL 1.1 (https://github.com/intel/intel-one-mono/blob/main/OFL.txt).

DRAFT

[TODO[MEDIUM]: make imports precise! We **want** to get into using dot notation as soon as possible!]

Note that **R2 (Context Lifting)** will be crucial to define the syntax.

4.1. Numbers.

[TODO[MEDIUM]: make this easier to use! Maybe **just** use binary or so? OR can formally define digits and nibbles later, if so we choose.]

We add numbers and provide equivalnces between them. Before this, we need an important construction: **general pairs**.

Welkin's main encoding uses binary words, but add notation for decimal and hexadecimal.

```
bit <--> 0 | 1,
digit <--> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9,
nibble <--> @digit | A | B | C | D | E | F
```

LISTING 1. Binary, decimal, and hexadecimal digits.

```
number <--> binary | decimal | hex,
decimal <--> {top --> digit, next --> decimal},
hex <--> {top --> nibble, next --> hex}
```

LISTING 2. Definition of words.

DRAFT

4.2. Encoding.

Welkin uses ASCII as its base encoding. The term ASCII is slightly ambiguous, as there are subtly distinct variants, so we formally define US-ASCII as a standard version.[9]

Definition 4.2.0. Welkin's encoding consists of **Printable US-ASCII**, listed in Table 3, as well as character **EOF**, with code 58. In Welkin (Listing 3):

```
encoding <--> {"EOF" --> 58, @printable}
```

LISTING 3. The full encoding for Welkin, written in Welkin.

To represent general encodings, there is a binary format supported for strings, see Listing 7.

Moreover, we will need the following sets of terminals.

```
PRINTABLE <--> {ENCODING, ~ENCODING.EOF},
WHITESPACE <--> "\t" | "\n" | "\r" | " ",
RESERVED <--> @DELIMITER | "*" | "\" | "@" | "#",
DELIMITER <--> "{" | "}" | "'"| "." | "," | "|"
```

LISTING 4. Important character classes.

4.3. Invertible Syntax Descriptions.

To more easily describe the language, we provide basic building blocks. Note that one of these, |, can be desugared in the bootstrap. For simplicity, we do not add many primitives. The names are taken from the the `parsec` combinator library in the programming language Haskell [LM01]. The authors use the following notation:

[9]Note that this table *itself* is a representation, which represents glyphs with binary words.

- `seq` to mean two rules that must be concatenated *directly*, no whitespace allowed.
- `lexeme`, to mean two rules can be joined with any number of whitespaces in between.
- `many_till`, to mean that in `A - many_till -> B`, parse zero or more instances of `A` until `B` is encountered.

In contrast to [LM01], however, we include the ability *print* as well or present the corresponding string. This is done through [RO10]. For the combinators we need, see Listing 5.

[TODO: an important thing is making sure that when using the standard library, invariants are held! So we may need verifier to come into play here!]

```
"TODO: indicate errors somehow!",
input <--> {} | {begin --> character, next --> input},

codec <--> {
  parse --> unit,
  print --> unit,

  {s --> input, u --> unit, ~{s - parse -> {}}} -->
  {
    {s - parse -> u}
    <-->
    {u - print -> s}
  }
},

"TODO: incorporate abstract syntax tree here! Important!",
{before - seq -> after} <--> {
  @codec,
  parse -->
  | "" --> {},
  | {c --> char} --> {name --> c}

  print -->
  | {} --> ""
  |
  ,
},

{before - many_till -> after} <--> {
  repeat <--> {} | before - seq -> repeat,
  repeat - seq -> after
},

{before - lexeme -> after} <--> {before - seq -> WHITESPACE - many_till
-> after}
```

LISTING 5. Definitions for the main combinators used.

Lemma 4.3.1. *In the combinators above,* `parser` *commutes with* `print`*. More precisely: ?.*

Proof.

□

DRAFT

4.4. **Character Classes.**

Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph
32	20	Space	56	38	8	80	50	P	104	68	h
33	21	!	57	39	9	81	51	Q	105	69	i
34	22	"	58	3A	:	82	52	R	106	6A	j
35	23	#	59	3B	;	83	53	S	107	6B	k
36	24	$	60	3C	<	84	54	T	108	6C	l
37	25	%	61	3D	=	85	55	U	109	6D	m
38	26	&	62	3E	>	86	56	V	110	6E	n
39	27	'	63	3F	?	87	57	W	111	6F	o
40	28	(	64	40	@	88	58	X	112	70	p
41	29	)	65	41	A	89	59	Y	113	71	q
42	2A	*	66	42	B	90	5A	Z	114	72	r
43	2B	+	67	43	C	91	5B	[	115	73	s
44	2C	,	68	44	D	92	5C	\	116	74	t
45	2D	-	69	45	E	93	5D	]	117	75	u
46	2E	.	70	46	F	94	5E	^	118	76	v
47	2F	/	71	47	G	95	5F	_	119	77	w
48	30	0	72	48	H	96	60	`	120	78	x
49	31	1	73	49	I	97	61	a	121	79	y
50	32	2	74	4A	J	98	62	b	122	7A	z
51	33	3	75	4B	K	99	63	c	123	7B	{
52	34	4	76	4C	L	100	64	d	124	7C	|
53	35	5	77	4D	M	101	65	e	125	7D	}
54	36	6	78	4E	N	102	66	f	126	7E	~
55	37	7	79	4F	O	103	67	g			

TABLE 3. Printable US-ASCII codes and glyphs.

We denote specific characters through quotes, escaping if necessary. There are several important character classes in Listing 4, denoted through double quotes.

```
MODULE <--> {"#" - seq -> ID},
IMPORT <--> {"@" - seq -> ID}
ID <--> {{@printable, ~{@reserved, @whitespace}} - many_till ->
@reserved | @whitespace}
```

LISTING 6. IDs.

Strings allow escaped single or double quotes, see Listing 7. IDs are special cases of strings that do not require quotes but forbid whitespace and certain characters, see Listing 6.

[TODO: clean up with `many_till`!]

DRAFT

```
STRING <--> SQ_STRING| DQ_STRING,
STRING_CONTENTS <--> SQ_CONTENTS | DQ_CONTENTS,

SQ_STRING <--> {start --> "'", contents --> sq_contents, end --> "'"},
DQ_STRING <--> {start --> "\"", contents --> DQ_CONTENTS, end --> "\""},

SQ_CONTENTS <--> {{@printable, ~{"\""}}| {"\" - seq -> "\" | "'"}, next
--> SQ_CONTENTS},
DQ_CONTENTS <--> {top --> "" | {@printable, ~{"\""}}| {"\" - seq -> "\"
| "\""}, next --> DQ_CONTENTS},
```

LISTING 7. Strings.

4.5. **The Welkin Grammar.**

Welkin's grammar is displayed in Listing 8, inspired by a minimal, C-style syntax.

```
start <--> {nodes - lexeme -> EOF},

terms <--> "" | {term - lexeme -> terms_tail},
terms_tail <--> {"," - lexeme -> term}
  | ","
  | "",

term <--> {
  node - seq ->
  {} | {
    arc - seq -> term
  }
},

arc <--> right_arc | other_arc
right_arc <--> {"-" - seq -> node - seq -> "->"}
other_arc <--> {"<-" - seq -> node - seq -> ("-" | "->")}

node {path - lexeme -> contents | {}}

path {MODIFIER | {} --> path_segment* - seq ->  unit}

path_segment <--> UNIT | ".*" | {@times, factor --> "."}

path_segment ::= UNIT | ".*" | "."+


UNIT <--> ID | STRING
```

LISTING 8. The grammar for Welkin. This includes the definitions of `id` and `string`, which are also defined in Listing 6 and Listing 7, respectively.

4.6. **Proof of Unambiguity.**

We show that, by construction, the combinators we used form an $\mathbb{L}(1)$ grammar. This is a special kind of grammar that can be efficiently parsed. We will keep definitions self-contained; for more background, consult [Aho+06], [RS70].

DRAFT

4.7. **Import Resolution.**

5. METATHEORY

[TODO: need a quick way to *establish* the main definitions for information, then *justify* here why they are complete.]

This section discusses the provably most general definition of information. This section is optional. For the base definition of information, refer to Definition 3.4.18.

1) We show that Welkin's base theory is equivalent to a weak fragment of arithmetic, $I\Delta_0$.
2) We overview Artemov's work in serial consistency. [Art24].
3) We establish the meta-theory, and show it can encompass any formal system.
4) We show the meta-theory "meta-proves" its own soundness, *without* contradicting known impossibility results.
5) We conclude with the definition of information.

For notation, we will write PA for Peano Arithmetic and PRA for Primitive Recursive Arithmetic. Moreover, without loss of generality, we will consider only first-order theories T that are RE. This can be shown in the proof of Traktenbroht's Theorem [Tra50]. In this proof, a step is to show that for any Turing machine M, there is a a first-order theory whose theorems concide with the language of M. [TODO: decide if we want to prove this is true IN $I\Delta_0$, to show that it alone is enough. Put into the appendix. Pretty simple embedding.]

5.1. **Establishing the Floor: $I\Delta_0$.**

To compare Welkin against other theories, we show the unit verify can be translated to $I\Delta_0$, a weak fragment of arithmetic, and vice versa. This subsection is optional, and we will keep the proofs at a high-level for readability. For background in first-order logic, please refer to [Men09].

Herein, let $a \Leftrightarrow b$ denote a if and only if b. Robinson Arithmetic denotes the base set of axioms; refer to [HP17], which use $I\Sigma_0$ to denote $I\Delta_0$.

[TODO[SMALL]: fix equation labels!]

Definition 5.1.0. Robinson Arithmetic Q is the first-order theory over the language of arithmetic with the following axioms, universally quantified over x, y, z:

- **Q1:** $\neg(S(x) \neq 0)$.
- **Q2:** $S(x) = S(y) \Rightarrow x = y$.
- **Q3:** $x \neq 0 \Rightarrow \exists y.x = S(y)$.
- **Q4:** $x + 0 = x$.
- **Q5:** $x + S(y) = S(x + y)$.
- **Q6:** $x * 0 = 0$.
- **Q7:** $x * S(y) = (x * y) + x$.
- **Q8:** $x \leq y \equiv \exists z.z + x = y$

DRAFT

Definition 5.1.1. The theory $I\Delta_0$ [PW81] consists of Q plus the **bounded induction schema**:

$$(\varphi(0) \wedge \forall x.(\varphi(x) \Rightarrow \varphi(x+1))) \Rightarrow \forall x.\varphi(x)$$

for each φ with bounded quantifiers, which means quantifiers $\exists x < t.\psi(x,t)$ and $\forall x < t.\psi(x,t)$ where x is free in term t and $\psi(x,t)$ is quantifier free.

Remark 5.1.2. Note that the induction schema is stronger than having open formulas. This allows statements about, e.g., odd and even numbers to be proved. We will need this to express $\mathrm{verifier}$.

[TODO[MEDIUM]: make this more rigorous.]

Lemma 5.1.3. *The unit* $\mathrm{verifier}$ *is definable in* $I\Delta_0$.

Proof Sketch. The claim relies on defining unit. From there, one can easily express the conditions in $\mathrm{verifier}$ by simple recursion.

To this end, we first argue that the inductive definitions can be written in $I\Delta_0$. Clearly, every handle can be expressed, indexing each triple of functions with Cantor's pairing function, sending triples $(\mathrm{UID}, \mathrm{RID}, \mathrm{HID})$ to natural numbers. Similarly, representations can be indexed by a pairing argument. It remains to show that blocks can be defined as well. We claim that an extended pairing function can be made that is defined inductively. [TODO: define this function!]

Second, it can be easily shown that each rule in Definition 3.1.12 are definable by induction, in at most 7 variables. □

An important consequence of this theorem is the following, proving that the metatheory for Welkin is as minimal as possible.

Now we proceed that Welkin's $\mathrm{verifier}$ is itself can process any $I\Delta_0$ proof.

Lemma 5.1.4. *There is a unit* Q *in Welkin and a bijection from propositions* φ *units* u_φ *such that* $Q \vdash \varphi$ *if and only if* $u_\varphi - \mathrm{part} \to \mathrm{Q}$.

Proof. Let Q be a new handle. Encode the naturals into words via Figure 2 and Definition 3.1.8, with successor represented through pairs. [TODO: define addition and multiplication via units, in the direct way.]

Axioms **Q1**-**Q3** easily follow from Lemma 3.2.3. The remaining axioms are included in Q. [TODO: note how quantifiers are correctly expressed in a separate lemma]. Using transitivity and ?, this completes the proof. □

Theorem 5.1.5. *Welkin's* $\mathrm{verifier}$ *verifies there is a context with all derivations of* $I\Delta_0$.

Proof. We prove that Q, constructed in Lemma 5.1.9, combined with the $\mathrm{verifier}$ unit proves the claim. TBD. □

Taken together, we can prove that Welkin has a minimal metatheory.

Corollary 5.1.6. (Base Theory: $I\Delta_0$). *Suppose* T *is another first order theory that proves the existence of Welkin's* $\mathrm{verifier}$. *Then* $T \Rightarrow I\Delta_0$.

Proof. Over $I\Delta_0$, Definition 5.1.21 proves the existence of $\mathrm{verifier}$ implies $I\Delta_0$. Thus, if T proves the existence of $\mathrm{verifier}$, it must satisfy $I\Delta_0$ as well, completing the proof. □

DRAFT

5.2. **Artemov's Approach.**

A major goal in Welkin is to express any representable notion, *including* any representabale proof. Proofs are a finite certificate that consists of a valid sequence of steps. Importantly, proofs can be verified using a *total* computable function, one that can check *any* given proof. Because of the requirement for computable checking, limitations in computable functions apply. These are illustrated by several impossibility theorems in the literature, including Gödel's infamous incompleteness theorems [Göd06].

However, previous impossibility results rely on proving strong properties of proofs. Artemov argues that weaker, constructive properties can be used instead. We provide a high-level definition of **serial-consistency** before proceeding onto the embedding in Welkin.

Definition 5.2.7. Suppose T is sufficiently strong to express its own proofs. Then T is **serial-consistent** if there is a **selector** for T. A **selector** is a computable function s [10] with two properties:

- s accepts any string that encodes a valid proof.
- if s accepts a proof, then it cannot contain a contradiction.

Artemov overcomes Gödel's second incompleteness theorem. This theorem states that Peano Arithmetic PA, providing basic properties of the natural numbers, cannot prove its own consistency. Artemov instead demonstrates that PA proves its own *serial-consistency*, as follows:

- He uses a weak meta-theory, in this case, Primitive Recursive Arithmetic (PRA). This theory is bounded by a weak principle of induction.
- Artemov encodes all proofs as natural numbers in PRA. PRA is sufficiently strong *just* to encode all of these proofs.
- Finally, he uses a proof by induction, over encoded proofs in PRA, to show that PA proves each *individual* proof contains no contradictions. This is possible through partial truth definitions.

To be clear: Artemov does *not* show that there is a *single* proof that PA is consistent, that works for all proofs. Instead, his method *takes* a proof as a parameter. There have been extensive discussion on the validity of this technique, and its acceptance by other logicians.[11]

Despite its initial controversy, Artemov's techniques follow closely with several constructive schools, see [Art24]. This thesis builds upon this result with a stronger property: **serial-soundness**. The exact same proof applies, using the same partial truth definition. Tarski's theorem says that a formal system cannot define its own truth predicate *at the object language* [Tar83]. In fact, this truth predicate is uncomputable, in general. We do not avoid Tarski's theorem, but we do approximate it *as* closely as partial computable functions allow. Because some logics may not have a notion of inconsistency, we will not define consistency itself. However, given the results in Section 3, first order logic can be expressed in several ways.

[10] Technically, this is a primitive recursive function, but we will quickly generalize this to *any* computable function.

[11] The discussion is available online at: https://mathoverflow.net/questions/469247/situation-with-artemovs-paper.

DRAFT

5.3. **Truth and Soundness.**

We embed "Convention T", or Tarski's criterion for truth [Tar83], as unit. Recall that, at this stage, Welkin deals with truh systems. The user's task is to ensure they accurately represent their referents, because these are free parameters in the theory (see Remark 3.1.2). For this reason, the aim is to demonstrate the *preservation* of these truth systems. This means preserving their axioms and rules of inference. This motivates the following definition.

[TODO: clarify how in works! May need to work with a direct representation.]

Definition 5.3.8. Let c be a context. A unit t is called a **truth predicate** for c if for every unit u, $u - c \to t$ if and only if $u - \text{in} \to c$.

Truth systems in Welkin, however, are separated by contexts. How can it be determined whether a unit is "sound", or truthful about derivations? Our approach is to *merge* the contents and determine if they match. This precisely uses rule ? for whether a representation is contained in a unit. [TODO: still need to clarify global context here! Also, ensure that .]

Definition 5.3.9. A unit u is **sound** if for every context c and representation $\{\{a - d \to b\} - \text{in} \to c\} - \text{in} \to u$, $\{@c, \{a - d \to b\}\} \longleftrightarrow c$.

In other words, adding the representation $\{a - d \to b\}$ does **not** add new information to c.

[TODO: discuss Tarski's undefinability theorem here.]

In this setting, Gödel's incompleteness theorems manifest in the undecidability of derivations. The second cannot be avoided with soundness outright, but it can with a weaker notion. With the *usual* notions of soundness, the second cannot be avoided. But it this can be mitigated using a weaker notion of soundness.

[TODO: define the equivalent of total computable functions for units! And determine if a **single** input is enough to trust soundness. If not, need to construct examples to show why!]

Definition 5.3.10. A unit u is **serial-sound** if there is a total computable function such that the following property holds: given an arbitrary, but fixed, unit v as input, verifies that any representation "claimed in u" is "actually in v". Moreover the selector must be proven **constructively**, or "without explosion"[TODO: define in general!].

Note that not every total computable function can be proven to be total. For this reason, we have a more technical definition in Section 5.4. For now, we can prove the following.

[TODO[SMALL]: make sure to clarify this proof!]

Lemma 5.3.11. *The unit* verifier *is serial-sound.*

Proof. Equivalent to Lemma 3.3.6. □

Moreover, serial-soundness is transitive, just as with soundness. This corresponds to composition of meta-proofs.

Lemma 5.3.12. *Let u, u', u'' be units, and suppose u is serial-sound. If u proves that u' is serial-sound and u' proves that u'' is serial sound, then u proves that u'' is serial sound.*

DRAFT

Proof. Suppose u is serial-sound. Then, in the context u, u contains the a selector of u'. This selector can then be composed with that from u'' proven in u' to complete the proof. □

5.4. **The Meta Unit.**

[TODO: complete! Want to ensure that proofs can be *chains* of serial-sound units. Moreover, should include *any* expressible derivation that is "reliable"!]

Definition 5.4.13. The **meta recursor** meta over all units is defined recursively:

- **Base case:** $\mathrm{verifier} - \mathrm{in} \to \mathrm{meta}$.
- **Recursive step:** for each proof in meta, adding a self-verifying unit is a valid proof step.

If meta verifies a claim C, then we say that meta **meta-proves** C.

Remark 5.4.14. A different approach to create more powerful chains of theories is *reflection*. One example is from Feferman [Fef62]: starting from PA, one can add the encoded statement, roughly meaning: "PA is sound". One can then iterate this through transfinite induction. While this, too, is a way to express all proofs, reflection is a *subset* of our techniques.

[TODO: make this constructive! Probably possible. And maybe instead of consistency we want *coherency*, to generalize having a false operator? So to prevent *all* things from being proved, or only useful things being proved?]

Theorem 5.4.15. *The unit* meta *meta-proves that serial soundness implies soundness.*

Proof. Let u be a serial sound unit. By way of contradiction, suppose u is not sound. Then there exists a specific instance φ that u contains but is not true. However, as u is finite, there is a finite trace that verifies the failure of soundness. But, because the serial soundness proof is constructive, it reliable proves that *no such* trace exists, a contradiction. Therefore, u must be sound. □

Remarkably, meta *also* meta-proves its own serial soundness.

Corollary 5.4.16. *The unit* meta *meta-proves its own serial soundness, and hence, soundness.*

Proof. Fix some claim C, and suppose C can be proved by a terminating sequence of self-verifying theories. On each C with a proof, define s to be one of these proofs. [TODO: make sure choice is not needed! Probably need a way to standardize this, though maybe it's already in the constructive definition? Or the existence of *any* of these is enough]. Clearly s is total as the sequence terminates, and is correct by construction. This proves meta is serial-sound. Thus, by Theorem 5.4.14, meta is sound □

5.5. **Proof of Complete Expressivity.**

[TODO: find a simpler proof! Maybe we don't *need* ordinals?]

To show that this covers *every* proof that can be computably recognized, we need to discuss recursive ordinals. We provide an embedding via unit. For more background on ordinals, refer to [Mon73], [Kle38]. To keep the presentation self-contained, we will define a unit for recursive ordinals. [TODO: maybe use a separate appendix?]

DRAFT

At a high level, it is not enough to carry out a *successor* step, and then solely combine theories through a union in the limit stage. At most, we can only reach the Feferman–Schütte ordinal, denoted Γ_0. [CITE!]. [TODO: discuss predicative and impredicative theories more closely!] Feferman himself *advocated* for the sole use of predicative mathematics [CITE]. However, the aim of Welkin is to be *as* expressive as possible, especially as impredicative theories have use in certain areas, e.g., formal methods projects [TODO: cite these!]

Due to Artemov, there is a *massive* jump from $I\Delta_0$ all the way to ZFC. This is because, over $I\Delta_0$ as the base-theory, ZFC is self-verifying (proves its own soundness. A reason for this power in ZFC is the axiom of replacement, or even comprehension. We extend this realization using hyperarithmetic sets, which are known to cover every recursive ordinal, see [Kle38] [TODO: make sure this citation is correct!]. Based on Kleene's theorem, our key construction in the limit case is to add this: let λ be a limit ordinal and consider all given theories $T_1, T_2, ..., T_\beta, ...$ with $\beta < \lambda$. Then $T_\lambda = \bigcup_{\beta<\lambda} T_\beta \cup \mathrm{Comp}_\lambda(\Delta_1^1)$. The set $\mathrm{Comp}_\lambda(\Delta_1^1)$ is an axiom schema over each proposition φ definable in the extended theory, stating that [TODO: clarify notation! And double check hyperarithmetic literature!]

$\exists X_{\varphi,\lambda}.\forall n.\Big(n \in X \Leftrightarrow \Big(\exists p.\ \mathrm{Prov}\Big(T_{\{\psi(p)\}}\Big), \varphi(p)\Big)\Big)$.

Note that this set is encoded through $I\Delta_0$ through an appropriate encoding. And the provability predicate is also encoded through the base theory. [TODO: clarify these encodings!] This new axiom naturally extends the Axiom of Comprehension through *other* sound theories.

[TODO: clarify that hyperarithmetic sets are connected to definability in Δ_1^1!]

We now prove a major property of Welkin.[TODO: definitely clean up!]

Theorem 5.5.17. *The unit* meta *expresses any computably expressible proof. More precisely, for any recursive ordinal* α*, there is a unit* $T \in \mathrm{meta}$ *with greater theoretic proof ordinal than* α*.*

Proof. Using only autonomous progressions [CITE!], it is already know that one can reach the Feferman-Schütte ordinal Γ_0. We may therefore focus on reaching higher ordinals, which depends on the limit case. But [Kle38] already proves that using hyperarithmetic sets in the successor ensures that we can find *some* $\beta > \alpha$. This completes the proof. □

Note, however, that detecting if an ordinal is recursive is undecidable. The upper bound is the best one can hope for, in general.

As an immediate corollary, this proves that unit, being a simple verifier, does *not* impose further restrictions on proofs. Moreover, additional layers of meta-theories are not needed, thanks to how expressive serial-soundness is.

Remark 5.5.18. The definition of information encompasses *every* meta-proof. Theorem 5.5.16 is to show that even *one* chain of theories is enough to majorize each recursive ordinal. And the construction is entirely definable by a Turing machine!

6. Conclusion

[TODO: update conclusion.]

DRAFT

This thesis introduced Welkin, a universal, formalized information language. The syntax (Section 4) was defined rigorously with a small EBNF, shown to be accepted by an LL(1) grammar, showing that parsing is unambiguous. The semantics (Section 3) were provided with several passes to convert parse trees into units, which contain both a hierarchical and relational structure for scoping and direct representations, respectively. Units have key properties that enable them to express any partial computable function Theorem 3.1.2, in conjungtion with expressing any truth management system, demonstrates **universality** of the system. This is practically demonstrated by showing that all the major paradigms in Information Management and Knowledge Management are expressible within Welkin. Finally, the bootstrap in Section 3 self-hosts the language within a bounded 64 variant, whose complete Unambiguity (as well as the grammar's prior) establishes **standardization**. Revisions further enhance this by

The remaining sections show several areas for future work. This list is not exhaustive and, by the previous arguments, and can be applied to *any* subject with computable representations (essentially, any human subject).

6.1. Comparisons to Existing Literature.

In addition to domain specific proposals, there are approaches for general IM which still fail to resolve all issues. One prominent example is Burgin's General Theory of Information [Bur09] that comprehensively includes many separate areas for IM, including the complexity-based Algorithmic Information Theory, through a free parameter called an "infological system", which encompasses domain specific terminology and concepts. In contrast to other approaches, Burgin's generalized theory is flexible and enables greater coverage of different kinds of information [Mik23]. Despite this coverage Burgin does not closely tie the free parameter with his formal analysis of Algorithmic Information Theory, making it unclear how to use this in a practical implementation. Burgin's Theory of Information, along with broad proposals, have severe shortcomings, highlighting major obstacles for IM.

6.2. Programming Languages and Formal Verification.

[TODO: add discussions on supporting interoperability with languages, providing more robust and unified implementations of core libraries, compilers, and operating systems.]

Taking the enable custom hardware implementations for checking and reduce the surface area for attacks on verifying certifications for many applications, including cryptography.

Moreover, the proposed architecture could use an LLM as an oracle.

6.3. Mathematical and Scientific Knowledge.

[TODO: discuss more applications of Welkin in depth.]

There are several possible projects to pursue in mathematics and scientific research. For mathematics, there are several existing rojects for storing mathematical information (see [CF09] for more details). Older proposals, including the QED Manifesto [KR16] and the Module system for Mathematical Theories (MMT), aimed to be more general and have seen limited success. More centralized systems, like `mathlib` in the Lean proof assistant [The20], have seen adoption but do not give equal coverage nor are

interoperable with other systems. Welkin enables this interopreeable through gradual translations, and with adaptive optimizations, one can always determine if there is enough *direct* information to complete a translation. This will help facilitate reusability among major tools, and aid in formal verification (Section 6.2) well.

Along with mathematics, Welkin could provide more rigorous frameworks for the sciences, with are currently scattered with different proposals. One prominent proposal is the Findable Accessible Interoperable Reusable (FAIR) guidelines [Wil+16]. Instead of providing a concrete specification or implementation, FAIR provides best practices for storing scientific information. However, multiple papers have outlined problems with these overarching principles, including missing checks on data quality [Gui+25], missing expressiveness for ethics frameworks [Car+21], and severe ambiguities that affect implementations [Jac+20]. Welkin addresses these by using contexts strategically. Experiments can be compared using revisions, and disagreements between experts can be analyzed using separate contexts. These contexts can then *distinguish* between different theories, and scientists can select the unit with the best or most comprehensive evidence. Metrics for such evidence can be *representable* to a certain point, but at a minimum, they can be more effectively analyzed.

6.4. **Humanities.**

[TODO: significantly expand.]

Information Management in the humanities has few models, including an adaption of FAIR [Har+20] and discipline specific, linked databases in the PARNTHEOS project [Hed+19]. Welkin could assist by providing a space to help standardize this and localize different publication styles and literary theories.

DRAFT

A Bootstrap Document: `welkin.welkin`

```
#welkin,

bit <--> 0 | 1,
word <--> {} | {top --> bit, next --> word},

handle <--> {MID | RID | SYM --> word},

equals <--> {
  0 <--> 0,
  1 <--> 1,
  ~{0 <--> 1},

  w1 | w2 --> word,
  {w1 <--> w2} <-->
    | {w1 <--> {} <--> w2}
    | {w1.top <--> w2.top, w1.next <--> w2.next},

  h1 | h2 --> handle,
  {h1 <--> h2} <--> {
    h1.MID <--> h2.MID,
    h1.RID <--> h2.RID,
    h1.SYM <--> h2.SYM,
  }
}

unit {
```

```
  {} --> .,
  u | v | c --> .,
  @u --> .,

  import_block <-->
    | {}
    | @u
    | {import_block, ~v}
    | {~v, import_block},
  import_block --> .,

  {u - c -> v} --> .,
}

part {
  u | v --> .unit,
  u --> {@u, v},
  ~{{@u, v} --> u},
  {@u, ~v} --> u,
  ~{u --> {@u, ~v}},
}

verify {
  query { context | goal --> ..unit },
  derivation --> .unit,

  derivation - verify -> query,
}

numeral {
  base {
    bit <--> ..bit,
    digit <--> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9,
    nibble <--> @decimal | A | B | C | D | E | F,
  },

  system {
    binary <--> ..word,
    decimal <--> {} | {top --> .base.digit | next --> decimal},
    hexadecimal <--> {} | {top --> .base.nibble | next --> decimal},
    hex <--> hexadecimal
  },

  convert {

  }
}

encoding {


}


codec <--> {
  parse --> unit,
  print --> unit,

  {s --> input, u --> unit, ~{s - parse -> {}}} -->
```

DRAFT

```
  {
    {s - parse -> u}
    <-->
    {u - print -> s}
  }
},

characters {


}

grammar {

}

information {

}


compress {


}
```

References

[Ack89] Ackoff, R L, From Data to Wisdom, Journal of Applied Systems Analysis **16** (1989) 3–9.

[Aho+06] Aho, Alfred V., Lam, Monica S., Sethi, Ravi, and Ullman, Jeffrey D., *Compilers: Principles, Techniques, and Tools (2nd Edition)*, Addison Wesley, 2006.

[Ang18] Angles, Renzo, The Property Graph Database Model., in *AMW* (Olteanu, Dan and Poblete, Barbara (eds.), CEUR-WS.org.

[AGH05] Arnold, Ken, Gosling, James, and Holmes, David, *The Java Programming Language*, 4th ed., The Java Series, Addison-Wesley Professional, Upper Saddle River, NJ, 2005.

[Art24] Artemov, Sergei, Serial properties, selector proofs and the provability of consistency, Journal of Logic and Computation **35** no. 3 (2024) exae34.

[Atk23] Atkin, Albert, Peirce's Theory of Signs, Spring2023 ed., Metaphysics Research Lab, Stanford University, 2023.

[Bat00] Bateson, Gregory, *Steps to an ecology of mind*, University of Chicago Press ed., University of Chicago Press, Chicago, 2000.

[Buc91] Buckland, Michael K., Information as thing, Journal of the American Society for Information Science **42** no. 5 (1991) 351–360.

[Bur09] Burgin, Mark, *Theory of Information*, World Scientific, 2009.

[BF95] Buvač, Saša, and Fikes, Richard, A declarative formalization of knowledge translation, in *Proceedings of the Fourth International Conference on Information and Knowledge Management*, Association for Computing Machinery, Baltimore, Maryland, USA, pp. 340–347.

[CF09] Carette, Jacques, and Farmer, William M., A Review of Mathematical Knowledge Management, in *Intelligent Computer Mathematics* (Carette, Jacques, Dixon, Lucas, Coen, Claudio Sacerdoti, and Watt, Stephen M. (eds.), Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 233–246.

[Car+21] Carroll, Stephanie Russo, Herczog, Edit, Hudson, Maui, Russell, Keith, and Stall, Shelley, Operationalizing the CARE and FAIR Principles for Indigenous Data Futures, Scientific Data **8** no. 1 (2021) 108.

[Cur30] Curry, H. B., Grundlagen der Kombinatorischen Logik, American Journal of Mathematics **52** no. 3 (1930) 509–536.

[Dic25] Dictionary, Oxford English, welkin, n., Oxford University Press, 2025.

DRAFT

[Fef62] Feferman, Solomon, Transfinite Recursive Progressions of Axiomatic Theories, The Journal of Symbolic Logic **27** no. 3 (1962) 259–316.

[Fin85] Fine, Kit, *Reasoning with Arbitrary Objects*, Blackwell, New York, NY, USA, 1985.

[Flo10] Floridi, Luciano, *Information : a very short introduction*, Oxford University Press, Oxford ; New York, 2010.

[GWW07] Goczyla, Krzysztof, Waloszek, Wojciech, and Waloszek, Aleksander, Contextualization of a DL Knowledge Base., p. .

[Gui+25] Guillen-Aguinaga, Miriam, Aguinaga-Ontoso, Enrique, Guillen-Aguinaga, Laura, Guillen-Grima, Francisco, and Aguinaga-Ontoso, Ines, Data Quality in the Age of AI: A Review of Governance, Ethics, and the FAIR Principles, Data **10** no. 12 (2025).

[Göd06] Gödel, Kurt, Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I, Monatshefte für Mathematik **149** no. 1 (2006) 1–29.

[Har90] Harnad, Stevan, The Symbol Grounding Problem, Physica D **42** (1990) 335–346.

[Har07] Harrington, Brian, ASKNet: Automated Semantic Knowledge Network., in *AAAI*, AAAI Press, pp. 1862–1863.

[Har+20] Harrower, Natalie, Quinn, Mary, Tóth-Czifra, Erzsébet, and others, *Sustainable and FAIR Data Sharing in the Humanities: Recommendations of the ALLEA E-Humanities Working Group*, Berlin, 2020.

[HS13] Hazarika, Shyamanta, and Sinha, Smriti, Formal concept analysis: current trends and directions, Artificial Intelligence Review **44** (2013) 1–40.

[Hed+19] Hedges, Mark, Stuart, David, Tzedopoulos, George, Bassett, Sheena, Garnett, Vicky, Giacomi, Roberta, and Sanesi, Maurizio, *Digital Humanities Foresight: The future impact of digital methods, technologies and infrastructures*, GOEDOC, Dokumenten-und Publikationsserver der Georg-August-Universität Göttingen, 2019.

[Hit21] Hitzler, Pascal, A review of the semantic web field, Commun. ACM **64** no. 2 (2021) 76–83.

[Hit+09] Hitzler, Pascal, Krötzsch, Markus, Parsia, Bijan, Patel-Schneider, Peter F., and Rudolph, Sebastian, *OWL 2 Web Ontology Language Primer*, 2009.

[HMU07] Hopcroft, John E., Motwani, Rajeev, and Ullman, Jeffrey D., *Introduction to Automata Theory, Languages, and Computation*, Third, Pearson/Addison Wesley, Boston, 2007.

[HP17] Hájek, Petr, and Pudlák, Pavel, *Metamathematics of First-Order Arithmetic*, Perspectives in Logic, Cambridge University Press, 2017.

[Jac+20] Jacobsen, Annika, Miranda Azevedo, Ricardo de, Juty, Nick, Batista, Dominique, Coles, Simon, Cornet, Ronald, Courtot, Mélanie, Crosas, Mercè, Dumontier, Michel, Evelo, Chris T., Goble, Carole, Guizzardi, Giancarlo, Hansen, Karsten Kryger, Hasnain, Ali, Hettne, Kristina, Heringa, Jaap, Hooft, Rob W.W., Imming, Melanie, Jeffery, Keith G., Kaliyaperumal, Rajaram, Kersloot, Martijn G., Kirkpatrick, Christine R., Kuhn, Tobias, Labastida, Ignasi, Magagna, Barbara, McQuilton, Peter, Meyers, Natalie, Montesanti, Annalisa, Reisen, Mirjam van, Rocca-Serra, Philippe, Pergl, Robert, Sansone, Susanna-Assunta, Silva Santos, Luiz Olavo Bonino da, Schneider, Juliane, Strawn, George, Thompson, Mark, Waagmeester, Andra, Weigel, Tobias, Wilkinson, Mark D., Willighagen, Egon L., Wittenburg, Peter, Roos, Marco, Mons, Barend, and Schultes, Erik, FAIR Principles: Interpretations and Implementation Considerations, Data Intelligence **2** nos. 1–2 (2020) 10–29.

[KBp] KBpedia, OpenCyc Resources.

[Kle38] Kleene, S. C., On Notation for Ordinal Numbers, The Journal of Symbolic Logic **3** no. 4 (1938) 150–155.

[KR16] Kohlhase, Michael, and Rabe, Florian, QED Reloaded: Towards a Pluralistic Formal Library of Mathematical Knowledge, Journal of Formalized Reasoning **9** (2016) 201–234.

[LM01] Leijen, Daan, and Meijer, Erik, Parsec: Direct Style Monadic Parser Combinators For The Real World, (2001) .

[Len95] Lenat, Douglas B, CYC: A large-scale investment in knowledge infrastructure, Communications of the ACM **38** no. 11 (1995) 33–38.

[Len+90] Lenat, Douglas B., Guha, R. V., Pittman, Karen, Pratt, Dexter, and Shepherd, Mary, Cyc: toward programs with common sense, Commun. ACM **33** no. 8 (1990) 30–49.

[The20] The mathlib Community, The lean mathematical library, in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, Association for Computing Machinery, New Orleans, LA, USA, pp. 367–381.

[McC93] McCarthy, John, Notes on formalizing context, in *Proceedings of the 13th International Joint Conference on Artifical Intelligence - Volume 1*, Morgan Kaufmann Publishers Inc., Chambery, France, pp. 555–560.

[Men09] Mendelson, Elliott, *Introduction to Mathematical Logic*, 5th ed., Chapman & Hall/CRC, 2009.

[Mes+15] Mesgari, Mostafa, Okoli, Chitu, Mehdi, Mohamad, Nielsen, Finn Årup, and Lanamäki, Arto, ``The sum of all human knowledge'': A systematic review of scholarly research on the content of Wikipedia, Journal of the Association for Information Science and Technology **66** no. 2 (2015) 219–245.

[Mik23] Mikkilineni, Rao, Mark Burgin's Legacy: The General Theory of Information, the Digital Genome, and the Future of Machine Intelligence, Philosophies **8** no. 6 (2023).

[Mon73] Monk, J. Donald, Introduction to Set Theory, Journal of Symbolic Logic **38** no. 1 (1973) 151.

[MDV26] Mora, Angel, Dominguez, Jesus, and Villalon, Ana, fcaR: Formal Concept Analysis, 2026.

[PW81] Paris, Jeff B., and Wilkie, Alex J., \Delta_0 Sets and Induction, in *Proceedings of Open Days in Model Theory and Set Theory* (Guzicki, W., Marek, W., Plec, A., and Rauszer, C. (eds.), Leeds University Press, Jadwisin, Poland, pp. 237–248.

[Qui37] Quine, W. V., New Foundations for Mathematical Logic, The American Mathematical Monthly **44** no. 2 (1937) 70–80.

[Rei17] Reisman, Miriam, EHRs: The Challenge of Making Electronic Data Usable and Interoperable, P & T : a peer-reviewed journal for formulary management **42** (2017) 572–575.

[RO10] Rendel, Tillmann, and Ostermann, Klaus, Invertible syntax descriptions: unifying parsing and pretty printing, in *Proceedings of the Third ACM Haskell Symposium on Haskell*, Association for Computing Machinery, Baltimore, Maryland, USA, pp. 1–12.

[RWE15] Robinson, Ian, Webber, Jim, and Eifrem, Emil, *Graph Databases*, 2nd ed., O'Reilly Media, Inc., 2015.

[RS70] Rosenkrantz, D.J., and Stearns, R.E., Properties of deterministic top-down grammars, Information and Control **17** no. 3 (1970) 226–256.

[SL17] Smith, Brent, and Linden, Greg, Two Decades of Recommender Systems at Amazon.com, IEEE Internet Computing **21** no. 3 (2017) 12–18.

[Str13] Stroustrup, Bjarne, *The C++ Programming Language*, 4th ed., Addison-Wesley Professional, Boston, MA, 2013.

[Tar83] Tarski, Alfred, The Concept of Truth in Formalized Languages, Hackett Publishing Co., Indianapolis, IN, 1983.

[Tra50] Trakhtenbrot, Boris Abramovich, The impossibility of an algorithm for the decidability problem on finite classes, Dokl.\ Akad.\ Nok.\ SSSR **70** no. 4 (1950) 569–572.

[Tur36] Turing, Alan M., On Computable Numbers, with an Application to the Entscheidungsproblem, Proceedings of the London Mathematical Society **2** no. 42 (1936) 230–265.

[Wil+16] Wilkinson, Mark D., Dumontier, Michel, Aalbersberg, IJsbrand Jan, Appleton, Gabrielle, Axton, Myles, Baak, Arie, Blomberg, Niklas, Boiten, Jan-Willem, Silva Santos, Luiz Olavo Bonino da, Bourne, Philip E., Bouwman, Jildau, Brookes, Anthony J., Clark, Tim, Crosas, Mercè, Dillo, Ingrid, Dumon, Olivier, Edmunds, Scott C., Evelo, Chris T. A., Finkers, Richard, González-Beltrán, Alejandra N., Gray, Alasdair J. G., Groth, Paul, Goble, Carole A., Grethe, Jeffrey S., Heringa, Jaap, Hoen, Peter A. C. 't, Hooft, Rob W. W., Kuhn, Tobias, Kok, Ruben G., Kok, Joost N., Lusher, Scott J., Martone, Maryann E., Mons, Albert, Packer, Abel Laerte, Persson, Bengt, Rocca-Serra, Philippe, Roos, Marco, Schaik, Rene C. van, Sansone, Susanna-Assunta, Schultes, Erik Anthony, Sengstag, Thierry, Slater, Ted, Strawn, George O., Swertz, Morris A., Thompson, Mark, Lei, Johan van der, Mulligen, Erik M. van, Velterop, Jan, Waagmeester, Andra, Wittenburg, Peter, Wolstencroft, Katy, Zhao, Jun, and Mons, Barend, The FAIR Guiding Principles for scientific data management and stewardship, Scientific Data **3** (2016).

[Zhu+23] Zhu, Xixi, Liu, Bin, Zhu, Cheng, Ding, Zhaoyun, and Yao, Li, Approximate Reasoning for Large-Scale ABox in OWL DL Based on Neural-Symbolic Learning, Mathematics **11** no. 3 (2023).

Department of Mathematics, University of Colorado at Boulder, Boulder, CO
Email address: oscar-bender-stone@protonmail.com

DRAFT